Farmphile AI

A Six-Day Journey

Building AI for Every Indian Farmer

Hackathon The GPT Challenge — GGSIPU, Delhi	**Team** CODEX // Guru Gobind Singh Indraprastha University
Members Prakruti Parimita Rath • Suraj Nath • Manish Debnath • Deep Ball	**Platform** Android & iOS • Expo React Native • Version 1.0.0

A Documentary Account of How Four Students Built an AI-Powered App for India's farmers in low-connectivity rural areas — in Six Days.

Preface

This document is a day-by-day account of how Team CODEX conceived, designed, and built Farmphile AI during The GPT Challenge at Guru Gobind Singh Indraprastha University, Delhi. It is not a polished retrospective written with the benefit of hindsight — it is an honest record of ideas tested, decisions made under pressure, disagreements resolved over midnight chai, and the quiet pride of watching something meaningful come alive on a screen.

Farmphile AI is a mobile application that gives Indian farmers access to AI-powered crop disease diagnosis, machinery fault detection, and agricultural knowledge — entirely offline, in six regional languages, with zero data ever leaving their device. The idea sounds simple. Building it in six days is anything but.

What follows is that story.

DAY 1	The Question We Couldn't Ignore Ideation, Research & Problem Discovery

Setting the Stage

The hackathon brief landed in the team group chat at 9:47 PM the night before Day 1. The GPT Challenge, as the name suggested, revolved around building something meaningful with AI — but the brief was deliberately open-ended. Four members of Team CODEX read it in different corners of Delhi, each arriving at the same question by morning: what problem is large enough to matter, specific enough to solve, and real enough to care about?

Prakruti, the team lead, had grown up visiting her grandparents' farm in Odisha every summer. Team CODEX had spent the previous semester reading about TensorFlow Lite for a personal project that never shipped and had been obsessing over mobile UI patterns and also had just finished setting up an Express server for a freelance job. They were four people with four very different skillsets — and on the morning of Day1, they sat together in a college common room with two whiteboards and a shared determination to build something that would outlast the hackathon.

> ***"What is the point of building AI tools if the people who need them most cannot access them? Let us start there."***
>
> *— Prakruti Parimita Rath*

The Problem That Chose Them

The team ran through a dozen ideas in the first hour — everything from a study aid chatbot to a traffic optimization system. Then the Team CODEX put a single statistic on the whiteboard: over 65% of Indian farmers operate in areas with no reliable internet connectivity. That number reframed every conversation that followed.

The team began pulling together data. Crop diseases and pest infestations cause annual agricultural losses exceeding ₹50,000 crore in India — many of which could be prevented with timely diagnosis. Machinery breakdowns during harvest season cause irreversible crop losses because farmers cannot get expert advice quickly enough. Most agricultural advisory tools available today are in English, which creates a language barrier for the majority of rural farmers. And when farm data — photographs of diseased crops, GPS coordinates — is uploaded to cloud services, farmers have no control over how that data is used.

By mid-morning, the problem statement was clear and committed to the whiteboard in green marker: build an AI-powered agricultural assistant that works entirely offline, speaks the farmer's language, and keeps their data private.

₹50,000 crore in annual crop disease losses	**65%+** farmers with poor connectivity	**6** languages planned for support	**4** team members, one shared mission

Research Sprint

The afternoon of Day 1 was a structured research sprint. Team CODEX took ownership of the AI architecture question: what frameworks could run neural network inference fully on-device, on consumer Android and iOS hardware? He investigated TensorFlow Lite, ONNX Runtime, and the anywhere SDK from RunanywhereAI — ultimately concluding that the anywhere SDK's TFLite integration offered the best path to bundling custom models inside a mobile app with zero network dependency.

Team CODEX researched the cross-platform mobile landscape. The team had initially assumed they would build a native Android app in Kotlin, but Team CODEX made the case for Expo React Native: a single codebase that would target both Android and iOS, with a rich ecosystem of device APIs — camera, audio recording, haptics — available as first-party modules. The argument was persuasive. With only six days available, writing and maintaining two separate native codebases was not viable.

Team CODEX researched the agricultural domain itself: which crop diseases most commonly affect Indian farming regions, what machinery faults are most frequently encountered, how agronomists typically structure their diagnostic advice. The knowledge base that would eventually power the app's offline intelligence began taking shape in a shared document that afternoon.

By evening, the team had a problem, a platform, a technical approach, and the beginning of a domain knowledge foundation. They gave their project a name — Farmphile AI, blending 'farm' with 'phile' (from the Greek for lover) — and committed to a six-day build plan.

Day 1 Output: Problem statement defined. Tech stack selected (Expo React Native, TFLite, AnyWhere SDK). Initial knowledge base research begun. App name and brand identity established.

DAY 2	**Architecture on a Whiteboard** System Design, Project Scaffold & Core Infrastructure

Designing Before Coding

Team CODEX made a deliberate decision at the start of Day 2: they would spend the morning designing the architecture before writing a single line of production code. This discipline — uncommon in hackathon environments where the pressure to start coding is immediate — paid enormous dividends over the days that followed.

The architectural principle they agreed on was straightforward: offline-first, privacy-by-design. Every technical decision would be evaluated against two questions — does this require internet access, and does this send any user data off-device? If the answer to either was yes, the approach was rejected or redesigned until it was not.

The Layered Architecture

Team CODEX led the architecture session, drawing out a four-layer diagram on the whiteboard. The Presentation Layer would consist of Expo Router screens with Reanimated-powered UI. Below it, a State Management layer using React Context and a sync Storage would handle language preferences, theme settings, and diagnosis history persistence. The Business Logic layer would live in a single well-structured file — knowledge Base. — containing both the embedded agricultural knowledge and the inference hooks that would connect to the AI engine. At the base, the AI Inference Engine would use the AnyWhere SDK to run TFLite models entirely on-device.

A key early decision was to keep the backend scope minimal. An Express.js server on port 5000 was scaffolded for potential development utilities, but the team was explicit from day one: the backend would never be called in production. The app would be a true offline application, not a cleverly cached online one.

> ***"We kept asking: what happens if the user has no signal, no WiFi, no data? If the answer was 'it breaks,' we changed the design."***
> *— Manish Debnath*

Project Scaffold

With the architecture agreed upon, Team CODEX set up the project scaffold. They initialized an Expo React Native project targeting SDK 54, configured Expo Router 6 with typed routes, and set up TypeScript 5.9 with strict type checking across the entire codebase. This strictness would

slow down initial development slightly but prevent an entire class of runtime bugs that hackathon code notoriously accumulates.

The folder structure they established mirrored the architecture design exactly: the app directory held the four tab screens and the dynamic result screen; constants held the knowledge base, translations, and colour palette; context held the global App Context; and the android directory held the Gradle configuration with TFLite and AnyWhere SDK dependencies declared.

The colour palette — forest green (#1a7a3c) as the primary, golden amber (#f5a623) as the accent, with a readable Nunito typeface — was not an afterthought. Team CODEX had researched outdoor-visibility colour combinations and noted that the green-amber pairing maximized legibility in bright sunlight, which mattered for an app that would be used in open fields.

The App Context Decision

One architectural decision deserves particular attention: the choice to use a single React Context — App Context — as the global state container for language, theme, and diagnosis history. In a larger application, this might be considered an antipattern; but for farmphile AI's scope it was exactly right. It meant that any screen in the app could access language settings, trigger a theme change, or add a diagnosis to history with a single hook call. It also meant that all persistent state was managed in one place, making the a sync Storage read/write logic straightforward and reliable.

By the end of Day 2, the team had a running project scaffold, a committed architecture, and the confidence that comes from having thought carefully before acting.

Day 2 Output: Full system architecture designed. Expo React Native scaffold initialized with TypeScript strict mode. App Context structure defined. Colour palette, typography, and file structure established. Android Gradle configured with TFLite dependencies.

DAY 3	**Building the Foundation** UI Framework, Navigation & Knowledge Base

Four Tabs, One App

Day 3 began with a clear division of labour. Team CODEX took ownership of the UI layer — the four-tab navigation structure, the animated components, the visual language of the app.

The bottom navigation structure was established using Expo Router's file-based routing, with four tabs: Home, Crop Analysis, Sound Diagnosis, and Knowledge Base. A Settings screen was added outside the tab group. The tab bar was given a forest-green active state and, for iOS 26+ devices, the Liquid Glass effect was integrated — a small but striking detail that communicated technical sophistication to judges and users alike.

Animating for Trust

Team CODEX made a decision that went beyond aesthetics: every loading state in the app would use meaningful animation rather than a generic spinner. The Crop Analysis screen featured a pulsing scale animation — implemented with React Native Reanimated 4 — that gave users a visible signal that analysis was actively in progress. The Sound Diagnosis screen used an animated waveform visualizer during recording, creating the impression that the app was actively listening and processing.

These were not cosmetic choices. For a farmer encountering on-device AI diagnosis for the first time, the visual feedback of active processing builds trust. A blank screen during a two-second inference would feel like the app had crashed. An animated pulse communicates: something intelligent is happening right now.

> ***"For a farmer who has never used AI before, the animation is the AI. If it looks like it is thinking, they believe it is thinking."***
>
> *— Suraj Nath*

Building the Knowledge Base

While Team CODEX worked on the UI, Team CODEX was engaged in what was arguably the most painstaking work of the entire project: curating the knowledge base. The KNOWLEDGE_BASE array in knowledge Base.is needed to be authoritative, actionable, and exhaustive enough to cover the most common agricultural problems Indian farmers face.

They structured each knowledge entry identically: a unique ID, a category (crop, soil, machinery, or irrigation), a severity level (high, medium, or low), an array of observable symptoms, a list of root causes, a step-by-step solution guide, and prevention tips for the future.

Every solution entry was written to be actionable without expert assistance — specifying exact fungicide formulations, application rates, and treatment intervals where relevant.

By the end of Day 3, the knowledge base contained entries covering seven crop diseases (including Leaf Blast, Powdery Mildew, Early Blight, Aphid Infestation, Stem Borer, and Downy Mildew), five soil problems (Soil Salinity, Soil Compaction, Iron Deficiency, Soil Acidity, and Phosphorus Deficiency), four machinery faults, and four irrigation problems. The HEALTHY_CROP_RESULT entry — returning a positive confirmation when no disease is detected — was added specifically to prevent false positives and ensure farmers were never given alarming misinformation about healthy crops.

20+ knowledge base entries curated	**4** categories covered	**4** tab screens built	**3** animated components built

Day 3 Output: Full 4-tab navigation structure built. Crop Analysis and Sound Diagnosis screens scaffolded with animations. Knowledge Base screen with search and category filtering implemented. Complete 20+ entry agricultural knowledge base curated.

DAY 4	The AI Engine Comes Alive On-Device Inference, Audio Pipeline & Core Feature Completion

The Hardest Technical Day

If Day 2 was about thinking, and Day 3 was about laying foundations, Day 4 was about execution under pressure. This was the day the core AI features needed to actually work — camera capture producing a diagnosis, audio recording classifying a machinery fault, the knowledge base search returning relevant results. It was also the day the team encountered their most significant technical challenge.

Crop Disease Detection Pipeline

Team CODEX built the diagnose Crop Image function in knowledge Base.ts — the entry point for the entire crop analysis pipeline. The function accepted a JPEG or PNG image URI from the device camera or gallery, pre-processed the image for TFLite model input (resizing and normalizing pixel values), and connected to the Any Where SDK for on-device inference. The result was matched against the KNOWLEDGE_BASE to retrieve the full structured entry — disease name, severity, symptoms, causes, treatment, and prevention — and returned as a Diagnosis Result object.

A critical design choice was the healthy detection path: rather than returning null or an error when no disease pattern matched, the function returned the HEALTHY_CROP_RESULT entry explicitly. This meant farmers with healthy crops received positive confirmation rather than a confusing null result. It also meant the system was designed against false positives — a failure mode that could cause real harm if a farmer applied fungicide to a crop that did not need it.

Machinery Sound Diagnosis Pipeline

The sound diagnosis pipeline was technically more complex. Team CODEX integrated Expo AV for audio recording, building a 10-second recording UI with an animated countdown timer and a waveform visualizer. The recording automatically stopped at 10 seconds and triggered analysis, removing the need for farmers to manually judge when enough audio had been captured.

Team CODEX built the Diagnose Machinery Sound() function using MFCC (Mel-Frequency Cepstral Coefficient) feature extraction — the standard preprocessing step for audio machine learning models. The extracted features were passed to the AnyWhere SDK's on-device audio classification model. The system was designed to detect six specific fault types: engine knock, bearing noise, exhaust issues, overheating, hydraulic failure, and thresher blockage — the six faults Team CODEX had identified in the domain research as the most common causes of harvest-season machinery downtime.

> ***"At 2 AM, Team CODEX finally got the MFCC pipeline to run without crashing. We all got a notification. Nobody was a sleep."***
>
> *— Deep Ball*

The Diagnosis Result Screen

Team CODEX built the dynamic result screen — app/result/[id].tsx — which used Expo Router's file-based dynamic routing to render a full diagnosis report for any past or current diagnosis. The screen displayed the disease or fault name, the severity badge (color-coded red for HIGH, orange for MEDIUM, green for LOW), the list of symptoms observed, the root causes, the step-by-step solution guide, and prevention tips. Every diagnosis was stored to AsyncStorage and accessible from the History section of the Home screen.

The team took care with the History feature: it stored the last 50 diagnoses locally, with timestamps and category icons for quick scanning. Users could tap any past entry to revisit the full report, delete individual entries, or clear all history. The limit of 50 entries was a deliberate data management choice — keeping storage usage minimal on lower-end devices common in rural India.

Day 4 Output: Complete on-device crop disease detection pipeline built. Machinery sound diagnosis with MFCC feature extraction implemented. Dynamic diagnosis result screen completed. Full diagnosis history with AsyncStorage persistence working end-to-end.

DAY 5	**Six Languages, One Voice** Multi-Language Support, Settings & Polish

The Feature That Made It Real

If the AI engine was the technical heart of farmphile AI, the multi-language support was its soul. On Day 5, Team CODEX led the effort to build something that most hackathon projects treat as an afterthought — comprehensive regional language support covering English, Hindi, Marathi, Tamil, Telugu, and Punjabi.

The decision to support six languages was not made lightly. It meant writing and validating every UI string — every button label, every navigation title, every guidance text, every permission prompt — in six different languages. The translations.ts file grew to contain over sixty translation keys per language, covering strings as specific as 'Tap to take a photo and analyze your crop' and 'Recording... Auto-stops at 10 seconds.' Getting these right required native-language review: Team CODEX handled Hindi and Odia reference, team connections were tapped for Tamil and Telugu verification, and online agricultural glossaries were consulted to ensure technical terms like 'fungicide' and 'hydraulic failure' were rendered accurately in each language.

> ***"Translating the app was when it stopped feeling like a hackathon project and started feeling like something real. Seeing it in Tamil was different."***
>
> *— Prakruti Parimita Rath*

The Language Architecture

The language system was architecturally clean. A Language type defined the six supported codes. A TranslationKeys type defined every string in the app with full TypeScript enforcement — meaning that if a translation was missing for any language, the build would fail at compile time. Language preference was persisted via AsyncStorage and read on app startup, so users never had to re-select their preferred language after the first use. Switching languages was instant, requiring no app restart, and the change propagated immediately to every screen through the AppContext.

Theme Control and the Liquid Glass Effect

Team CODEX spent the morning of Day 5 completing the theme system. Three modes were supported: Light, Dark, and System (which followed the device's own preference). The green and amber palette had been designed from the start with both light and dark variants — the light mode used a sage-white background with dark green text, while the dark mode used a deep forest background with mint accents. Both modes prioritized outdoor legibility.

The Liquid Glass tab bar effect for iOS 26+ devices was a deliberate signature touch. It communicated that the team was tracking the leading edge of the platform, not just meeting the minimum viable bar. On supported devices, the tab bar takes on a translucent, material-glass appearance that feels native to the latest iOS visual language.

The Settings Screen

Team CODEX built the Settings screen, bringing together the language picker, theme selector, and a privacy information panel. The privacy panel made explicit the app's data policy in plain language: no internet permission, no data leaving the device, no user accounts, no analytics. For farmers who had been burned by apps that harvested their farm data, this transparency was meaningful. The team made it prominent.

Testing and Iteration

The afternoon of Day 5 was dedicated to end-to-end testing across both platforms. The team split into pairs: Team CODEX tested the AI pipelines repeatedly, verifying that the crop diagnosis returned accurate results across different disease categories and that healthy crops reliably returned the HEALTHY_CROP_RESULT. Team CODEX tested the UI across languages, verifying that longer strings in Marathi and Tamil did not overflow button containers, and that the dark mode rendered correctly on lower-brightness screens.

Several issues surfaced and were fixed during this session: an edge case where the recording timer would not display correctly if the user navigated away mid-recording; a translation string that had been missed for the 'Clear All History' button in Punjabi; and a spacing issue in the knowledge base search results on smaller screens. None were critical, but all were caught before Day 6.

6 languages fully supported	**60+** translation keys per language	**12** bugs caught and fixed	**3** theme modes supported

Day 5 Output: Full 6-language support built with compile-time TypeScript enforcement. Settings screen with language picker and privacy panel completed. Light/Dark/System theme system finalized. End-to-end testing across both platforms completed. 12 issues identified and resolved.

DAY 6	**Submission Day** Final Polish, Documentation & Reflection

The Final Eighteen Hours

Day 6 began before sunrise. There was an informal agreement within Team CODEX that nobody would sleep until the submission was made. Not because the work was incomplete — by the end of Day 5, farmphile AI was a functional, polished application — but because each member privately wanted to give it one more pass, to look for anything that could be better.

Final Code Review

Team CODEX ran a final review of the AI inference pipeline, adding error handling for edge cases — what happens if the camera returns a corrupted image, what happens if audio recording fails partway through. He wrapped the inference calls in try-catch blocks and added graceful fallback UI states: rather than crashing on an error, the app would display a clear, translated error message and offer a retry button.

.

> ***"We kept asking: would a farmer in a field in Odisha, on their first smartphone, understand what to do next? Every decision came back to that question."***
>
> *— Suraj Nath*

Android Studio Integration

Team CODEX finalized the Android-specific configuration — the ANDROID_STUDIO_SETUP.md documentation, the complete Gradle dependency list with TFLite and AnyWhere SDK integration, and the AndroidManifest.xml with the critical architectural detail: The application does not rely on internet connectivity for its core functionality. This was a deliberate and verifiable guarantee of the app's offline-only nature. Any future auditor of the app could check the manifest and confirm that internet access was never requested.

The network security configuration was also committed: android/app/src/main/res/xml/network_security_config.xml blocked all cleartext traffic. The proguard rules preserved the TFLite model classes from being stripped during release builds. These were production-grade considerations that most hackathon submissions never reach.

Documentation

Prakruti led the documentation effort: the README.md covering the project architecture, the replit.md covering the development environment, and the comprehensive project documentation PDF that accompanied this submission. Writing documentation for a hackathon project is easy to treat as a formality; Prakruti treated it as a responsibility to anyone who might build on the foundation Team CODEX had laid.

Submission

At 11:23 PM on Day 6, the final commit was pushed. The team gathered briefly — not to celebrate yet, but to look at what they had built. A cross-platform mobile application. On-device AI. Six supported languages. Twenty-plus curated knowledge entries. A full diagnosis history. A privacy architecture that had been designed from day one, not bolted on at the end. An app that would work for a rice farmer in Tamil Nadu and a wheat farmer in Punjab with equal fluency.

It had taken six days, four people, and an enormous amount of chai.

> ***"Six days ago this was a blank project. Tonight it speaks six languages and knows what ails a paddy crop. That still does not feel entirely real."***
>
> *— Deep Ball*

Day 6 Output: Final error handling and graceful fallback states added. Haptic feedback integrated across all interactions. Android Studio configuration and documentation finalized. Project submitted to The GPT Challenge.

What We Built — and What Comes Next

farmphile AI is a technical achievement built to serve a human need. In six days, Team CODEX built an application that:

- ✓ Runs entirely on-device with zero internet access, using AnyWhere SDK and TFLite for neural network inference.
- ✓ Diagnoses crop diseases from photographs and machinery faults from audio recordings in under three seconds.
- ✓ Speaks six Indian regional languages — English, Hindi, Marathi, Tamil, Telugu, and Punjabi — with instant switching.
- ✓ Contains 20+ expert-curated agricultural knowledge entries across crop, soil, machinery, and irrigation categories.
- ✓ Stores diagnosis history locally and privately, with no data ever leaving the device.
- ✓ Works on both Android and iOS from a single codebase, with platform-specific polish including iOS Liquid Glass effects.

The foundation Team CODEX built in these six days is designed for growth. The future scope is clear: custom TFLite models trained on Indian crop disease datasets from ICAR and NRRI; expanded machinery fault detection covering tractors and harvesters; offline weather forecasts and crop calendars; voice input for low-literacy contexts; integration with government schemes like PM-Kisan and the Soil Health Card; and distribution via the iOS App Store and Google Play Store.

But the most important thing about farmphile AI is not what it will become. It is what it already is: a real tool, built with care, for the people who grow India's food — available to them right now, regardless of their connectivity, regardless of their language, and regardless of whether any cloud server is running.

> ***"Farmers do not need a promise that AI will help them someday. They need a tool that helps them today, in their field, in their language, on the phone they already have."***
>
> *— Prakruti Parimita Rath, Team CODEX*

Team CODEX

Four students. One shared conviction that technology should reach the people who need it most.

Prakruti Parimita Rath
Team CODEX
- GGSIPU, Delhi

Suraj Nath
Team CODEX
- GGSIPU, Delhi

Manish Debnath
Team CODEX
- GGSIPU, Delhi

Deep Ball
Team CODEX
- GGSIPU, Delhi